

# Atelier 1 : diagnostic initial

ShopFlow · PostgreSQL 18.6 · résultats et interprétation

**Contexte.** PostgreSQL 18.6 (Docker postgres:18, WSL2), base et schéma shopflow, chargés par 01\_schema.sql et 02\_donnees.sql : 1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes. work\_mem = 4 MB, shared\_buffers = 128 MB. Aucun index ajouté : seuls ceux des contraintes (clés primaires, UNIQUE). Mesures à chaud, le 05/10/2026.

## 1. Les deux requêtes

**Q1 : historique du client 42** (page d'API, très sélective)

```
SELECT id, created_at, total
FROM shopflow.commandes
WHERE client_id = 42
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

**Q2 : indicateurs mensuels** (lit toute la table)

```
SELECT date_trunc('month', created_at) AS mois,
       statut,
       count(*) AS nb_commandes,
       sum(total) AS chiffre_affaires
FROM shopflow.commandes
GROUP BY mois, statut
ORDER BY mois, statut;
```

## 2. Résultats fonctionnels vérifiés

On vérifie l'exactitude avant de mesurer : ces chiffres sont la référence qui doit rester identique, au centime, après toute optimisation (slides 6 et 24).

| Req. | Lignes     | Total           | Contrôle de référence                                                       | OK |
|------|------------|-----------------|-----------------------------------------------------------------------------|----|
| Q1   | 20         | 14 400,00 €     | Le client 42 a 100 commandes (47 760,00 €), la page en garde 20             | ✓  |
| Q2   | 30 groupes | 29 775 001,25 € | sum(total) sur toute la table : 100 000 commandes, 29 775 001,25 €. Écart 0 | ✓  |

## 3. Mesures : 5 répétitions après échauffement

3 exécutions d'échauffement, puis 5 avec EXPLAIN (ANALYZE, BUFFERS). Execution Time en ms. Les 5 plans de chaque requête sont identiques.

| Req. | #1    | #2    | #3    | #4    | #5    | Médiane      | Buffers                 |
|------|-------|-------|-------|-------|-------|--------------|-------------------------|
| Q1   | 0,207 | 0,267 | 0,234 | 0,208 | 0,229 | <b>0,229</b> | hit=99                  |
| Q2   | 84,3  | 84,1  | 80,5  | 86,2  | 78,7  | <b>84,1</b>  | hit=1 680, temp 380/381 |

### Froid ou chaud

Première exécution de Q2, à froid : 1 096,8 ms (shared read=1330). À chaud : ≈ 84 ms (shared read=0). Même plan, 13 fois plus rapide : l'état du cache doit toujours être précisé (slide 11).

## 4. Nœud coûteux et hypothèse

### Q2 : agrégation mensuelle

| Nœud (lecture de bas en haut)               | Temps propre   | Part        |
|---------------------------------------------|----------------|-------------|
| Seq Scan on commandes (100 000 lignes)      | ≈ 25 ms        | 33 %        |
| <b>Sort : external merge, Disk 3 040 kB</b> | <b>≈ 37 ms</b> | <b>48 %</b> |
| GroupAggregate (30 groupes)                 | ≈ 15 ms        | 20 %        |

**Nœud coûteux : le Sort.** Il trie 100 000 lignes pour produire 30 groupes et déborde sur disque (381 pages temporaires). Le Seq Scan est normal, puisque toute la table est nécessaire.

**Hypothèse :** work\_mem (4 MB) est trop petit pour ce tri, **et** le planificateur estime 80 048 groupes au lieu de 30, faute de statistique sur date\_trunc('month', created\_at). Il choisit donc de trier plutôt que de faire un HashAggregate en mémoire. **Testé ensuite sur une copie :** SET work\_mem = '32MB' et une statistique sur l'expression, voir « Optimisations testées » (hypothèse confirmée, gain ≈ ×1,7 à ×1,9).

### Q1 : historique du client 42

Déjà rapide (≈ 0,23 ms) et l'estimation est exacte (100 lignes prévues, 100 lues). L'accès passe par l'index unique (client\_id, cle\_idempotence) issu d'une contrainte. **Nœud le plus coûteux : le Bitmap Heap Scan**, qui lit 88 pages pour 100 commandes dispersées, puis un tri n'en garde que 20. **Hypothèse :** le coût croît avec le nombre de commandes du client, parce que l'index ne contient pas created\_at. Testé à l'Atelier 3 : un index sur (client\_id, created\_at DESC, id DESC) (×2,5 sur la requête de l'API, WAL +35 % à l'écriture).

#### Ce que j'ai compris de l'atelier 1

- 1. Je vérifie avant de mesurer.** Je compte les lignes et je calcule le total, puis je les compare à une référence calculée autrement. Sans ça, je pourrais accélérer une requête qui renvoie un mauvais résultat sans m'en rendre compte.
- 2. Une mesure ne suffit pas.** Je chauffe la requête puis je la lance 5 fois et je regarde la médiane. La même requête a pris 1 096 ms à froid et 84 ms à chaud : sans échauffement, je comparerais deux situations différentes.
- 3. Je lis le plan de bas en haut.** Chaque ligne est une étape (un nœud). Le temps d'un parent contient déjà celui de ses enfants : pour le coût propre du Sort, je fais  $62,6 - 25,4 \approx 37$  ms, et je n'additionne pas.
- 4. Le coût n'est pas toujours là où on l'attend.** Dans l'agrégation, la lecture de la table est normale (il faut tout lire). Le vrai coût est le tri, qui dépasse les 4 Mo de work\_mem et écrit sur le disque.
- 5. Je compare estimé et réel.** PostgreSQL prévoit 80 048 groupes alors qu'il y en a 30 : il se trompe parce qu'il n'a pas de statistique sur date\_trunc, et il choisit de trier plutôt que de compter en mémoire.
- 6. Un diagnostic complet** dit quelle étape coûte, pourquoi, et comment le vérifier. Je n'ai rien modifié ici : l'atelier interdit les index, les pistes sont à tester ensuite.

# Atelier 2 : une réécriture équivalente

Mêmes données, mêmes règles : on prouve l'équivalence du résultat avant de comparer les durées

**Principe (slide 31).** Réécrire une requête n'est valable que si le résultat pertinent est identique. Une version plus rapide qui perd ou duplique des lignes est **rejetée**. Cas de test choisis : le **client 42** (100 commandes) et la **commande 42** (3 lignes).

## A. Jointure qui duplique les clients, contre EXISTS

**Question métier :** quels clients ont au moins une commande payée ? Une ligne du résultat doit identifier **un client** (slides 24 et 27).

### Avant : jointure (A1)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
JOIN shopflow.commandes c ON c.client_id = cl.id
WHERE c.statut = 'payee';
```

### Après : EXISTS (A3)

```
SELECT cl.id, cl.nom
FROM shopflow.clients cl
WHERE EXISTS (SELECT 1 FROM shopflow.commandes c
              WHERE c.client_id = cl.id AND c.statut = 'payee');
```

| Variante               | Lignes | Clients distincts | Verdict                                                |
|------------------------|--------|-------------------|--------------------------------------------------------|
| A1 jointure            | 80 000 | 800               | <b>Rejetée</b> : le client 42 apparaît 100 fois        |
| A2 jointure + DISTINCT | 800    | 800               | Correcte, mais construit 80 000 lignes puis déduplique |
| A3 EXISTS              | 800    | 800               | Correcte, une ligne par client                         |

Contrôle du contenu : la différence symétrique entre A2 et A3 (EXCEPT dans les deux sens) renvoie **0 ligne**. Les deux listes de clients sont donc identiques.

| Variante      | #1   | #2   | #3   | #4   | #5   | Médiane     |
|---------------|------|------|------|------|------|-------------|
| A1 jointure   | 27,8 | 27,4 | 27,6 | 28,4 | 26,4 | <b>27,6</b> |
| A2 + DISTINCT | 49,2 | 40,2 | 35,3 | 37,7 | 42,7 | <b>40,2</b> |
| A3 EXISTS     | 11,3 | 13,3 | 11,7 | 11,3 | 10,8 | <b>11,3</b> |

Durées en ms, 5 répétitions après 3 exécutions d'échauffement.

### Interprétation

**A3 est la plus rapide (≈ 11 ms contre ≈ 27 ms) et la seule à répondre directement à la question.**

PostgreSQL la transforme en Nested Loop Semi Join : pour chaque client, il s'arrête dès qu'il trouve une commande payée (0,8 ligne lue en moyenne par client). La jointure construit 80 000 lignes (Hash Join après un Seq Scan de toute la table). Avec DISTINCT, il faut en plus un HashAggregate : c'est la plus lente. Une sous-requête corrélée n'implique donc pas une exécution complète par ligne (slide 34, question 6).

## B. Total par commande sans multiplier commandes.total

**Piège (slide 24) :** joindre commandes à lignes donne une ligne par **ligne de commande**, pas par commande. Chaque commande a 3 lignes, donc `sum(c.total)` compte chaque total 3 fois.

### Avant : faux (B1)

```
SELECT c.id, sum(c.total) AS total          -- FAUX
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

### Après : somme des lignes (B2)

```
SELECT c.id, sum(l.qte * l.prix_unitaire) AS total  -- CORRECT
FROM shopflow.commandes c
JOIN shopflow.lignes l ON l.commande_id = c.id
WHERE c.client_id = 42
GROUP BY c.id;
```

| Contrôle (client 42)        | Nb cmd | Total        | Verdict                         |
|-----------------------------|--------|--------------|---------------------------------|
| Référence : commandes.total | 100    | 47 760,00 €  | —                               |
| B1 sum(c.total)             | 100    | 143 280,00 € | <b>Rejetée</b> : total × 3      |
| B2 sum(qte × prix_unitaire) | 100    | 47 760,00 €  | Correcte : égale à la référence |

Exemple sur la commande 42 : B1 donne **720,00 €** (3 × 240), B2 donne **240,00 €** (58,75 + 80,00 + 101,25), et commandes.total vaut 240,00 €. Comparaison commande par commande de B2 avec commandes.total : **0 écart** sur les 100 commandes.

| Variante     | #1   | #2   | #3   | #4   | #5   | Médiane     |
|--------------|------|------|------|------|------|-------------|
| B1 (faux)    | 1,15 | 1,10 | 1,12 | 1,10 | 1,01 | <b>1,10</b> |
| B2 (correct) | 2,21 | 2,01 | 2,02 | 2,17 | 2,05 | <b>2,05</b> |

Durées en ms, 5 répétitions après échauffement.

#### Interprétation

**B1 est plus rapide (≈ 1,1 ms contre ≈ 2,1 ms), mais son résultat est faux : on la rejette.** B1 n'a besoin que de la clé de la ligne : PostgreSQL utilise un Index Only Scan sur lignes, sans lire la table. B2 doit lire qte et prix\_unitaire, donc un Index Scan avec accès à la table, puis un Sort avant le regroupement. Le surcoût de B2 (≈ 1 ms) est le prix de l'exactitude : on ne choisit pas une requête uniquement sur sa durée (slide 6).

## Conclusion de l'atelier

1. **EXISTS** remplace avantageusement la jointure quand on veut un résultat par client : correct et 2,4 fois plus rapide ici.
2. Pour un total par commande, on **agrège à la bonne granularité** (les lignes), jamais commandes.total après une jointure un-à-plusieurs.
3. Dans les deux cas, l'équivalence a été prouvée par un contrôle de contenu **avant** de comparer les temps. Les plans des variantes sont en annexe.

### Ce que j'ai compris de l'atelier 2

**1. Réécrire, c'est changer la forme, pas la réponse.** Je dois le prouver (mêmes clients, même total) avant de regarder les durées. Pour A, la différence symétrique entre les deux versions vaut 0 ligne.

**2. Une jointure avec une table « plusieurs » multiplie les lignes.** Un client a 100 commandes : il apparaît 100 fois (80 000 lignes pour 800 clients). Une commande a 3 lignes : son total est compté 3 fois (720 au lieu de 240). Je me demande toujours : « qu'est-ce qu'une ligne de mon résultat ? ».

**3. Pour savoir si quelque chose existe, j'utilise EXISTS.** Pas de doublons, et PostgreSQL s'arrête dès qu'il trouve : 11 ms contre 27 ms pour la jointure. Condition : je veux un résultat par client, pas une ligne par commande.

**4. DISTINCT ne corrige pas le problème, il le masque.** Il construit quand même 80 000 lignes puis les supprime : 40 ms, la version la plus lente.

**5. Plus rapide ne veut pas dire meilleur.** B1 est presque deux fois plus rapide (1,10 ms contre 2,05 ms) mais son total est triplé : je la rejette. B2 coûte 1 ms de plus parce qu'elle lit la table, et c'est le prix de l'exactitude.

**6. Je somme à la bonne granularité :** les lignes de commande, jamais commandes.total après une jointure un-à-plusieurs.

## Optimisations testées : le bilan

Chaque optimisation est mesurée avec le même protocole, avec un résultat identique à l'original

**Cadre.** L'atelier 1 interdit d'ajouter un index ou une statistique dans la base du laboratoire. Les pistes ont donc été testées sur une **copie** (shopflow\_opt, mêmes 100 000 commandes), **un seul changement à la fois**, avec 3 exécutions d'échauffement puis 5 mesures (médiane). La base shopflow du laboratoire n'a pas été touchée. **Équivalence prouvée :** pour chaque variante, l'empreinte md5 du résultat complet est identique à celle de la requête d'origine.

### Q2 : agrégation mensuelle (le problème était le tri qui déborde sur le disque)

| Optimisation                                               | Ce qui change dans le plan                                                                              | Médiane | Gain                          |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------|-------------------------------|
| Référence (copie)                                          | Sort external merge sur disque ; estimation 80 738 groupes                                              | 84,6 ms | —                             |
| <b>O1</b> : SET work_mem = '32MB'                          | HashAggregate en mémoire ; le tri ne porte plus que sur 30 lignes ; estimation toujours fausse (80 738) | 50,2 ms | ×1,7                          |
| <b>O2</b> : CREATE STATISTICS sur date_trunc, puis ANALYZE | Estimation corrigée : 30 groupes, HashAggregate de 40 kB                                                | 45,5 ms | ×1,9                          |
| O1 + O2                                                    | Même plan que O2                                                                                        | 47,9 ms | ×1,8 (pas mieux que O2 seule) |

**Mon hypothèse de l'atelier 1 est confirmée.** Les deux causes jouaient chacune : donner plus de mémoire ou corriger l'estimation suffit à remplacer le gros tri par un HashAggregate. **O2 corrige la cause** (PostgreSQL se trompait sur le nombre de groupes), **O1 contourne le symptôme** (le tri n'avait plus à déborder). Les deux gains sont du même ordre ( $\approx 1,7$  à  $1,9$  fois). **Coût** : O1 augmente la mémoire autorisée par opération de tri ou de hachage et par connexion : à réserver à une session ou une requête précise, pas à tout le serveur. O2 a un coût faible (une statistique à maintenir lors des ANALYZE), mais je ne l'ai pas mesuré. **Recommandation : O2.**

### Q1 : historique du client 42

L'optimisation de cette requête (index) est traitée dans l'**Atelier 3**, plus bas : mesurée sur la vraie table, avec la requête exacte de l'API, plusieurs clients, un client très actif et le coût d'écriture. Les mesures préliminaires faites sur la copie y sont remplacées.

## Gains obtenus par réécriture (atelier 2), sans aucun coût d'index

| Réécriture                                                       | Avant                             | Après                | Gain       |
|------------------------------------------------------------------|-----------------------------------|----------------------|------------|
| Clients ayant une commande payée : EXISTS au lieu de la jointure | 27,6 ms (80 000 lignes, doublons) | 11,3 ms (800 lignes) | ×2,4       |
| Total par commande : somme des lignes au lieu de commandes.total | total faux (× 3)                  | total correct        | exactitude |

### Ce que j'ai compris des optimisations

**1. Optimiser, c'est mesurer un gain ET un coût, avec un résultat inchangé.** Chaque ligne du tableau donne les trois.

**2. On corrige d'abord la cause.** La statistique règle l'erreur d'estimation à l'origine du mauvais plan ; augmenter work\_mem masque le symptôme.

**3. Un index n'est pas gratuit.** Il accélère la lecture mais ajoute du travail à chaque insertion (Atelier 3 : WAL +35 % pour l'index composé) et occupe de la place (≈ 30 % de la table).

**4. Je ne compare que dans le même environnement.** La copie est physiquement plus compacte que la base du laboratoire (840 pages contre 1 674 pour commandes) : les gains O1 et O2 se lisent au sein de la copie.

# Atelier 3 : l'historique client

Quel index pour l'historique d'un client, et à quel prix ? Comparaison mesurée de quatre états de la base

**Consigne (slide 12).** Comparer la base initiale, un index simple sur `client_id`, l'index composé et sa variante couvrante, sur plusieurs clients dont le 42, comparer un lot d'insertions, expliquer le gain ou l'absence de gain, et livrer un tableau de décision. **Documentation complète, scripts et mesures brutes : dossier atelier3/.**

**Protocole.** Les variantes sont créées sur la vraie table du laboratoire puis supprimées une par une (jamais deux ensemble). 3 échauffements puis 5 mesures par requête et par client (75 combinaisons), plus une **mesure renforcée en alternance** (51 tours, une table de travail par variante) car les durées sub-milliseconde sont bruitées : la même requête a pris 0,23 ms à l'atelier 1 et  $\approx$  0,55 ms aujourd'hui. **Le laboratoire a été vérifié avant et après : il a retrouvé exactement son état initial** (6 index, aucune statistique). Résultat fonctionnel identique (empreinte md5) pour les 75 combinaisons, les 51 tours et le client très actif.

## A. Les requêtes testées : le SQL réellement émis par l'application

Slide 13 : la compatibilité d'un index se teste avec le SQL émis par l'application. J'ai donc lu `01_server/api/commandes.mjs` et testé, en plus de la requête du cours, la requête exacte de l'API (première page, 21 lignes) et sa page suivante par curseur.

```
SELECT id::text AS id,
       to_char(created_at AT TIME ZONE 'UTC', 'YYYY-MM-DD"THH24:MI:SS.US"Z"') AS created_at,
       statut, total::text AS total
FROM shopflow.commandes AS commandes
WHERE client_id = 42
ORDER BY commandes.created_at DESC, commandes.id DESC      -- qualificatif indispensable (voir F)
LIMIT 21;
```

La page suivante ajoute `AND (created_at, id) < ('<date de la 20e ligne>', <id>)`.

## B. Les variantes (index créés puis supprimés)

| Variante                 | Définition                                                                       | Taille   | Part de la table |
|--------------------------|----------------------------------------------------------------------------------|----------|------------------|
| Index unique existant    | ( <code>client_id</code> , <code>cle_idempotence</code> ), issu de la contrainte | 1 864 kB | 13,9 %           |
| Simple                   | ( <code>client_id</code> )                                                       | 688 kB   | 5,1 %            |
| Composé                  | ( <code>client_id</code> , <code>created_at</code> DESC, <code>id</code> DESC)   | 3 984 kB | 29,7 %           |
| Couvrant (statut, total) | composé INCLUDE (statut, total)                                                  | 5 792 kB | 43,2 %           |
| Couvrant (total)         | composé INCLUDE (total)                                                          | 4 864 kB | 36,3 %           |

Table commandes : 100 000 lignes 1 674 pages 13 392 kB. Chaque client a exactement 100 commandes.

## C. Lecture pour le client 42 (51 tours en alternance, médiane en ms)

| Variante                 | Plan (requête API)         | Buffers | Cours | API   | Curseur | Gain API     |
|--------------------------|----------------------------|---------|-------|-------|---------|--------------|
| Initiale                 | Bitmap Heap Scan + Sort    | 96      | 0,546 | 0,690 | 0,690   | —            |
| Simple (client_id)       | Bitmap Heap Scan + Sort    | 96      | 0,548 | 0,729 | 0,707   | ≈ ×1 (aucun) |
| Composé                  | Index Scan, sans Sort      | 27      | 0,161 | 0,274 | 0,342   | ×2,5         |
| Couvrant (statut, total) | Index Only Scan, sans Sort | 7       | 0,094 | 0,199 | 0,201   | ×3,5         |
| Couvrant (total)         | Index Scan, sans Sort      | 27      | 0,086 | 0,267 | 0,303   | ×2,6         |

**Index simple : aucun gain.** La base a déjà un index qui commence par client\_id (l'index unique). PostgreSQL utilise le nouvel index, plus petit (688 kB contre 1 864 kB), mais **le plan est identique** : il retrouve les commandes du client sans pouvoir les donner dans l'ordre, lit les 100 et les trie pour en garder 20. **Composé** : l'égalité sur client\_id borne le groupe, puis l'index donne déjà l'ordre demandé : 21 entrées lues, plus de Sort. **Couvrant (statut, total)** : toutes les colonnes sont dans l'index, donc plus aucune lecture de table (7 buffers). **Couvrant (total)** : bon pour la requête du cours, mais l'API renvoie aussi statut : il faut retourner à la table, donc **aucun gain de plus que le composé** pour le SQL réellement émis.

## D. Cinq clients (protocole du cours, 5 mesures, requête API, médiane en ms)

| Variante                 | Client 1 | Client 42 | Client 250 | Client 500 | Client 1000 | Buffers  |
|--------------------------|----------|-----------|------------|------------|-------------|----------|
| Initiale                 | 0,799    | 0,786     | 0,803      | 0,884      | 0,771       | 99 à 109 |
| Simple (client_id)       | 0,766    | 0,847     | 0,701      | 0,771      | 0,769       | 99 à 109 |
| Composé                  | 0,256    | 0,321     | 0,281      | 0,358      | 0,273       | 27       |
| Couvrant (statut, total) | 0,160    | 0,161     | 0,172      | 0,168      | 0,203       | 7        |
| Couvrant (total)         | 0,297    | 0,257     | 0,309      | 0,276      | 0,269       | 27       |

Les 75 combinaisons (5 variantes × 3 requêtes × 5 clients) renvoient exactement le même résultat que la base initiale. Tous les clients ayant 100 commandes, les écarts entre clients sont du bruit : le jeu ne contient aucun client très actif.

## E. Un client très actif (client 7 porté à 20 100 commandes, requête API)

| Variante                 | Médiane  | Buffers | Gain   |
|--------------------------|----------|---------|--------|
| Initiale                 | 23,3 ms  | 287     | —      |
| Simple (client_id)       | 23,3 ms  | 287     | —      |
| Composé                  | 0,256 ms | 27      | ×91,1  |
| Couvrant (statut, total) | 0,189 ms | 7       | ×123,3 |
| Couvrant (total)         | 0,276 ms | 27      | ×84,5  |

**Sans index adapté, le coût croît avec le nombre de commandes du client** : 96 buffers pour 100 commandes, 287 pour 20 100, 23 ms. Avec le composé, PostgreSQL lit toujours 21 entrées : le coût **ne dépend plus du volume**. C'est l'argument principal du choix : à 100 commandes le gain est de 0,4 ms, il devient de 23 ms pour un client actif.

## F. Le piège de l'alias dans ORDER BY

| Variante                 | API qualifiée | API sans qualificatif | Buffers | Plan sans qualificatif          |
|--------------------------|---------------|-----------------------|---------|---------------------------------|
| Initiale                 | 0,690 ms      | 0,877 ms              | 96 / 93 | Bitmap Heap Scan + Sort         |
| Composé                  | 0,274 ms      | 0,783 ms              | 27 / 93 | Sort, index composé non utilisé |
| Couvrant (statut, total) | 0,199 ms      | 0,382 ms              | 7 / 11  | Index Only Scan puis Sort       |

Les colonnes de sortie de l'API s'appellent aussi id et created\_at (des textes formatés). Dans ORDER BY, un nom sans qualificatif désigne d'abord la colonne **de sortie** : le tri porterait sur les textes et aucun index ne peut le fournir. C'est pourquoi l'API écrit commandes.created\_at. Le résultat est le même, mais **l'index composé ne sert plus à rien** (0,783 ms, 93 buffers, comme sans index). Un bon index peut être rendu inutile par une réécriture anodine du SQL.

## G. Coût d'écriture : 20 000 insertions dans une transaction annulée (15 tours en alternance)

| Variante                 | Index | WAL (octets) | Écart WAL | Enreg. WAL | Durée médiane |
|--------------------------|-------|--------------|-----------|------------|---------------|
| Initiale                 | 2     | 4 623 674    | —         | 60 322     | 126,8 ms      |
| Simple (client_id)       | 3     | 5 927 578    | +28,2 %   | 80 327     | 180,9 ms      |
| Composé                  | 3     | 6 240 760    | +35,0 %   | 80 322     | 194,0 ms      |
| Couvrant (statut, total) | 3     | 6 570 048    | +42,1 %   | 80 322     | 219,0 ms      |
| Couvrant (total)         | 3     | 6 400 760    | +38,4 %   | 80 323     | 187,8 ms      |

**Le volume de WAL est la mesure fiable.** Chaque index ajouté ajoute environ 20 000 enregistrements, un par ligne insérée (60 322 = 3 × 20 000 + 322 avec la table, la clé primaire et l'index unique ; ≈ 80 322 avec un index de plus). **Les durées sont très bruitées** : sur trois exécutions complètes du protocole, la surcharge de l'index composé a varié de +23 % à +62 %. Je retiens donc : tout index ajoute du travail à chaque insertion, de l'ordre de **+30 à +40 % de WAL** ici, et plus l'index est large, plus c'est cher.

## H. Un index couvrant garantit-il zéro accès à la table ? (slide 9)

| Étape (table jetable)                                    | Plan            | Heap Fetches | Buffers |
|----------------------------------------------------------|-----------------|--------------|---------|
| Après VACUUM                                             | Index Only Scan | 0            | 4       |
| Après UPDATE des 100 commandes du client 42, sans VACUUM | Index Only Scan | <b>40</b>    | 48      |
| Après un nouveau VACUUM                                  | Index Only Scan | 0            | 8       |

**Non.** Dès que des lignes sont modifiées, PostgreSQL doit visiter la table jusqu'au prochain VACUUM. Sur la vraie table Heap Fetches vaut 0 parce qu'elle était entièrement « visible » avant l'atelier. Les 40 visites pour 20 lignes renvoyées s'expliquent probablement par les anciennes versions encore présentes dans l'index (hypothèse non vérifiée).

## I. Tableau de décision

| Critère                         | Simple        | Composé       | Couvrant (statut, total) | Couvrant (total)        |
|---------------------------------|---------------|---------------|--------------------------|-------------------------|
| Supprime le Sort                | non           | <b>oui</b>    | oui                      | slide : oui ; API : non |
| Gain API, client 42             | aucun         | ×2,5          | ×3,5                     | ×2,6                    |
| Gain, client à 20 100 commandes | aucun         | ×91           | ×123                     | ×84                     |
| Résultat identique              | oui           | oui           | oui                      | oui                     |
| Taille de l'index               | 688 kB        | 3 984 kB      | 5 792 kB                 | 4 864 kB                |
| Surcoût d'écriture (WAL)        | +28 %         | +35 %         | +42 %                    | +38 %                   |
| Dépend de VACUUM                | non           | non           | oui                      | oui                     |
| <b>Décision</b>                 | <b>Rejeté</b> | <b>Retenu</b> | <b>Option</b>            | <b>Rejeté</b>           |

**Pourquoi le composé et pas le couvrant ?** Le couvrant gagne encore 0,075 ms par appel (0,274 → 0,199 ms), soit moins que le gain du composé lui-même (0,416 ms). En échange il est 45 % plus gros, coûte plus en écriture et perd de son intérêt quand la table est modifiée. **Pourquoi rejeter le couvrant (total) ?** Il est inutile pour le SQL réellement émis. **Conditions de remise en cause :** si les créations de commandes dominent nettement, retirer l'index ; si l'endpoint devient un point chaud avec peu de mises à jour, envisager le couvrant. La charge réelle est inconnue : la décision est argumentée, pas définitive.

**Migration retenue (testée sur une base jetable : montée, rejeu sans erreur, contrôle de validité, retour arrière)**

```
-- 001 UP (labo)
SET lock_timeout = '5s';
CREATE INDEX IF NOT EXISTS idx_hist_client
  ON shopflow.commandes (client_id, created_at DESC, id DESC);
-- Déploiement : CREATE INDEX CONCURRENTLY, hors BEGIN/COMMIT, puis contrôle de indisvalid.

-- 001 DOWN (retour arrière)
DROP INDEX IF EXISTS shopflow.idx_hist_client;
```

La migration n'est **pas appliquée** au laboratoire (l'atelier demande une décision, et la base sert aux ateliers suivants). Les scripts (laboratoire, déploiement, vérification, retour arrière) sont dans atelier3/migration/.

### Ce que j'ai compris de l'atelier 3

- 1. Un index ne sert que s'il fournit ce que la requête demande.** L'index simple n'a rien changé : la base avait déjà un index commençant par client\_id, et aucun ne donnait l'ordre. Le plan est resté identique (96 buffers, Sort). Une absence de gain est un résultat, pas un échec à cacher.
- 2. L'ordre des colonnes suit l'usage.** client\_id d'abord (l'égalité isole le groupe), puis created\_at DESC, id DESC (l'ordre demandé) : PostgreSQL lit 21 entrées et s'arrête, au lieu de lire tout le groupe puis de trier.
- 3. Je teste le SQL réellement émis.** L'API renvoie statut et qualifie son ORDER BY. Un INCLUDE (total) convient à la requête du cours mais pas à l'API ; un ORDER BY sans qualificatif rend l'index inutile.
- 4. Le coût d'une requête sans index grandit avec le volume.** 96 buffers pour 100 commandes, 287 pour 20 100 ; avec l'index il reste constant. C'est ce qui justifie l'index bien plus que le gain à 100 commandes.
- 5. Un index se paie à chaque écriture.** WAL +35 % pour le composé, +42 % pour le couvrant. Les durées étant bruitées, je m'appuie sur le WAL, qui est déterministe.
- 6. INCLUDE ne garantit pas zéro accès à la table.** Heap Fetches passe de 0 à 40 après une modification sans VACUUM.
- 7. Je décide avec les chiffres et je dis ce qui me ferait changer d'avis.** Composé retenu, couvrant en option, simple et couvrant (total) rejetés ; et je garde l'index unique de la contrainte, qui ne se supprime pas pour « faire du ménage ».

# Questions de compréhension du cours

Réponses avec un exemple et une condition d'application, comme le demande la slide 33.

## 1. Pourquoi ne faut-il pas additionner les durées de tous les nœuds ?

Parce que le temps d'un parent **inclut** celui de ses enfants : on compterait le même travail plusieurs fois. Exemple : dans l'agrégation, le Sort affiche 62,6 ms mais le Seq Scan en dessous finit à 25,4 ms ; le coût propre du Sort est  $\approx 37$  ms. Condition : on soustrait la fin de l'enfant, sauf si le nœud tourne en boucle (loops > 1), où les valeurs sont des moyennes par boucle.

## 2. Quand un parcours séquentiel peut-il être raisonnable ?

Quand la table est petite, ou quand la requête a besoin d'une grande part des lignes. Exemple : l'agrégation mensuelle lit les 100 000 commandes : le Seq Scan est le bon choix, un index n'apporterait rien. Condition : ça ne tient plus si on cherche peu de lignes dans une grande table (client 42 : 100 commandes sur 100 000).

## 3. Pourquoi une moyenne faible peut-elle masquer un problème ?

Parce que quelques requêtes très lentes sont noyées dans beaucoup de rapides. Le p95 et le p99 montrent cette queue. Exemple : la slide 8 donne (valeurs fictives du cours) p50 = 80 ms mais p99 = 610 ms. Condition : un p95 n'est fiable qu'avec assez d'observations : avec mes 5 mesures, je ne regarde que la médiane.

## 4. Qu'est-ce qui rend deux benchmarks comparables ?

Mêmes données, même requête, même résultat attendu, même machine, même charge, et un état de cache documenté. On change un facteur à la fois. Exemple : Q2 à froid (1 096 ms) et à chaud (84 ms) ne sont pas comparables entre elles. Condition : ne comparer que des mesures faites dans le même état.

## 5. Pourquoi la jointure incidents / incident\_events change-t-elle la granularité ?

Une jointure un-à-plusieurs donne une ligne par élément du côté « plusieurs ». Exemple chez moi : clients JOIN commandes donne une ligne par commande (80 000), pas par client (800), et commandes JOIN lignes fait compter chaque total 3 fois. Condition : pour garder un résultat par parent, on utilise EXISTS, on agrège avant de joindre, ou on somme au niveau des lignes.

## 6. Une sous-requête corrélée produit-elle toujours une exécution par ligne ?

Non : le planificateur peut la transformer en semi-jointure. Exemple : mon EXISTS (A3) a donné un Nested Loop Semi Join qui s'arrête dès qu'une commande payée est trouvée (11 ms, plus rapide que la jointure). Condition : on le vérifie dans EXPLAIN (ANALYZE, BUFFERS), on ne le suppose pas.

## Annexe 1 : plans EXPLAIN (ANALYZE, BUFFERS) complets (mesure n°5)

### Q1 : historique du client 42

```
Limit (cost=329.84..329.89 rows=20 width=22) (actual time=0.183..0.186 rows=20.00 loops=1)
  Buffers: shared hit=99
  -> Sort (cost=329.84..330.09 rows=100 width=22) (actual time=0.182..0.184 rows=20.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 27kB
    Buffers: shared hit=99
    -> Bitmap Heap Scan on commandes (cost=5.07..327.18 rows=100 width=22) (actual
        time=0.039..0.147 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=88
      Buffers: shared hit=93
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
          rows=100 width=0) (actual time=0.022..0.023 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=5

Planning:
  Buffers: shared hit=125
Planning Time: 0.485 ms
Execution Time: 0.229 ms
```

### Q2 : agrégation mensuelle

```
GroupAggregate (cost=13279.82..15730.54 rows=80048 width=54) (actual time=54.107..77.900 rows=30.00
 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=1680, temp read=380 written=381
  -> Sort (cost=13279.82..13529.82 rows=100000 width=20) (actual time=53.828..62.580 rows=100000.00
 loops=1)
    Sort Key: (date_trunc('month'::text, created_at)), statut
    Sort Method: external merge Disk: 3040kB
    Buffers: shared hit=1680, temp read=380 written=381
    -> Seq Scan on commandes (cost=0.00..2924.00 rows=100000 width=20) (actual time=0.760..25.395
 rows=100000.00 loops=1)
      Buffers: shared hit=1674

Planning:
  Buffers: shared hit=152
Planning Time: 0.490 ms
Execution Time: 78.664 ms
```

## Annexe 2 : plans de l'atelier 2 (mesure n°5)

### A1 : jointure

```
Hash Join (cost=32.50..3168.22 rows=80317 width=18) (actual time=0.986..23.875 rows=80000.00 loops=1)
  Hash Cond: (c.client_id = cl.id)
  Buffers: shared hit=1684
  -> Seq Scan on commandes c (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.721..11.527
        rows=80000.00 loops=1)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20000
        Buffers: shared hit=1674
  -> Hash (cost=20.00..20.00 rows=1000 width=18) (actual time=0.249..0.250 rows=1000.00 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 58kB
        Buffers: shared hit=10
        -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual time=0.009..0.095
              rows=1000.00 loops=1)
              Buffers: shared hit=10
Planning:
  Buffers: shared hit=248
Planning Time: 0.655 ms
Execution Time: 26.422 ms
```

### A2 : jointure + DISTINCT

```
HashAggregate (cost=3569.81..3579.81 rows=1000 width=18) (actual time=42.510..42.599 rows=800.00
  loops=1)
  Group Key: cl.id, cl.nom
  Batches: 1 Memory Usage: 89kB
  Buffers: shared hit=1684
  -> Hash Join (cost=32.50..3168.22 rows=80317 width=18) (actual time=1.102..27.041 rows=80000.00
        loops=1)
        Hash Cond: (c.client_id = cl.id)
        Buffers: shared hit=1684
        -> Seq Scan on commandes c (cost=0.00..2924.00 rows=80317 width=8) (actual time=0.695..12.879
              rows=80000.00 loops=1)
              Filter: (statut = 'payee'::text)
              Rows Removed by Filter: 20000
              Buffers: shared hit=1674
        -> Hash (cost=20.00..20.00 rows=1000 width=18) (actual time=0.387..0.389 rows=1000.00 loops=1)
              Buckets: 1024 Batches: 1 Memory Usage: 58kB
              Buffers: shared hit=10
              -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual
                    time=0.008..0.220 rows=1000.00 loops=1)
                    Buffers: shared hit=10
Planning:
  Buffers: shared hit=173
Planning Time: 0.617 ms
Execution Time: 42.705 ms
```

### A3 : EXISTS

```
Nested Loop Semi Join (cost=0.29..568.98 rows=1000 width=18) (actual time=0.363..10.758 rows=800.00
  loops=1)
  Buffers: shared hit=21707
  -> Seq Scan on clients cl (cost=0.00..20.00 rows=1000 width=18) (actual time=0.008..0.081
        rows=1000.00 loops=1)
        Buffers: shared hit=10
  -> Index Scan using commandes_client_id_cle_idempotence_key on commandes c (cost=0.29..9.92 rows=80
        width=8) (actual time=0.010..0.010 rows=0.80 loops=1000)
        Index Cond: (client_id = cl.id)
        Filter: (statut = 'payee'::text)
        Rows Removed by Filter: 20
        Index Searches: 1000
        Buffers: shared hit=21697
Planning:
  Buffers: shared hit=248
Planning Time: 0.662 ms
Execution Time: 10.832 ms
```

## B1 : sum(c.total), faux

```
HashAggregate (cost=767.18..768.43 rows=100 width=40) (actual time=0.911..0.930 rows=100.00 loops=1)
  Group Key: c.id
  Batches: 1 Memory Usage: 56kB
  Buffers: shared hit=394
  -> Nested Loop (cost=5.49..765.68 rows=300 width=14) (actual time=0.055..0.803 rows=300.00 loops=1)
    Buffers: shared hit=394
    -> Bitmap Heap Scan on commandes c (cost=5.07..327.18 rows=100 width=14) (actual
        time=0.040..0.156 rows=100.00 loops=1)
      Recheck Cond: (client_id = 42)
      Heap Blocks: exact=88
      Buffers: shared hit=93
      -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
          rows=100 width=0) (actual time=0.027..0.027 rows=100.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=5
    -> Index Only Scan using lignes_pkey on lignes l (cost=0.42..4.35 rows=3 width=8) (actual
        time=0.006..0.006 rows=3.00 loops=100)
      Index Cond: (commande_id = c.id)
      Heap Fetches: 0
      Index Searches: 100
      Buffers: shared hit=301

Planning:
  Buffers: shared hit=179
Planning Time: 0.593 ms
Execution Time: 1.007 ms
```

## B2 : sum(lignes), correct

```
GroupAggregate (cost=1854.26..1859.26 rows=100 width=40) (actual time=1.877..1.995 rows=100.00 loops=1)
  Group Key: c.id
  Buffers: shared hit=696
  -> Sort (cost=1854.26..1855.01 rows=300 width=18) (actual time=1.866..1.880 rows=300.00 loops=1)
    Sort Key: c.id
    Sort Method: quicksort Memory: 36kB
    Buffers: shared hit=696
    -> Nested Loop (cost=5.49..1841.92 rows=300 width=18) (actual time=0.060..1.757 rows=300.00
        loops=1)
      Buffers: shared hit=693
      -> Bitmap Heap Scan on commandes c (cost=5.07..327.18 rows=100 width=8) (actual
          time=0.037..0.167 rows=100.00 loops=1)
        Recheck Cond: (client_id = 42)
        Heap Blocks: exact=88
        Buffers: shared hit=93
        -> Bitmap Index Scan on commandes_client_id_cle_idempotence_key (cost=0.00..5.04
            rows=100 width=0) (actual time=0.023..0.024 rows=100.00 loops=1)
          Index Cond: (client_id = 42)
          Index Searches: 1
          Buffers: shared hit=5
      -> Index Scan using lignes_pkey on lignes l (cost=0.42..15.12 rows=3 width=18) (actual
          time=0.010..0.015 rows=3.00 loops=100)
        Index Cond: (commande_id = c.id)
        Index Searches: 100
        Buffers: shared hit=600

Planning:
  Buffers: shared hit=182
Planning Time: 0.558 ms
Execution Time: 2.051 ms
```

## Annexe 3 : plans des optimisations O1 et O2 (copie, mesure n°5)

### Q2 référence (copie)

```
GroupAggregate (cost=12445.82..14906.89 rows=80738 width=54) (actual time=54.555..78.490 rows=30.00 loops=1)
  Group Key: (date_trunc('month'::text, created_at)), statut
  Buffers: shared hit=846, temp read=380 written=381
  -> Sort (cost=12445.82..12695.82 rows=100000 width=20) (actual time=54.310..63.319 rows=100000.00 loops=1)
    Sort Key: (date_trunc('month'::text, created_at)), statut
    Sort Method: external merge Disk: 3040kB
    Buffers: shared hit=846, temp read=380 written=381
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.014..24.472 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.350 ms
Execution Time: 79.261 ms
```

### Q2 avec O1 : work\_mem = 32MB

```
Sort (cost=10881.60..11083.45 rows=80738 width=54) (actual time=50.533..50.535 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=846
  -> HashAggregate (cost=3090.00..4301.07 rows=80738 width=54) (actual time=50.148..50.475 rows=30.00 loops=1)
    Group Key: date_trunc('month'::text, created_at), statut
    Batches: 1 Memory Usage: 2089kB
    Buffers: shared hit=840
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.019..26.247 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=136
Planning Time: 0.329 ms
Execution Time: 51.517 ms
```

### Q2 avec O2 : statistique sur date\_trunc

```
Sort (cost=3091.19..3091.26 rows=30 width=54) (actual time=45.145..45.147 rows=30.00 loops=1)
  Sort Key: (date_trunc('month'::text, created_at)), statut
  Sort Method: quicksort Memory: 26kB
  Buffers: shared hit=846
  -> HashAggregate (cost=3090.00..3090.45 rows=30 width=54) (actual time=45.101..45.108 rows=30.00 loops=1)
    Group Key: date_trunc('month'::text, created_at), statut
    Batches: 1 Memory Usage: 40kB
    Buffers: shared hit=840
    -> Seq Scan on commandes (cost=0.00..2090.00 rows=100000 width=20) (actual time=0.015..23.250 rows=100000.00 loops=1)
      Buffers: shared hit=840
Planning:
  Buffers: shared hit=147
Planning Time: 0.407 ms
Execution Time: 45.225 ms
```

## Annexe 4 : plans de l'atelier 3, requête API, client 42 (vraie table, mesure n°5)

Ces plans sont ceux d'une **seule mesure** (la n°5 de chaque série). Leur durée est bruitée et ne sert pas à comparer (ici le plan « Simple » affiche 1,306 ms alors que sa médiane sur 51 tours est de 0,729 ms) : ce qu'ils prouvent, ce sont les **nœuds** et les **buffers**, qui sont stables. Les médianes comparables sont dans la section C.

### Initiale

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z") AS
    created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit (cost=331.38..331.43 rows=21 width=118) (actual time=0.674..0.678 rows=21.00 loops=1)
  Buffers: shared hit=99
  -> Sort (cost=331.38..331.63 rows=100 width=118) (actual time=0.673..0.675 rows=21.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 30kB
    Buffers: shared hit=99
    -> Bitmap Heap Scan on commandes (cost=5.07..328.68 rows=100 width=118) (actual
        time=0.116..0.619 rows=100.00 loops=1)
        Recheck Cond: (client_id = 42)
        Heap Blocks: exact=88
        Buffers: shared hit=93
        -> Bitmap Index Scan on commandes_client_id_client_idempotence_key (cost=0.00..5.04
            rows=100 width=0) (actual time=0.029..0.030 rows=100.00 loops=1)
            Index Cond: (client_id = 42)
            Index Searches: 1
            Buffers: shared hit=5
Planning:
  Buffers: shared hit=137
Planning Time: 0.409 ms
Execution Time: 0.711 ms
```

### Simple (client\_id)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z") AS
    created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit (cost=331.38..331.43 rows=21 width=118) (actual time=1.247..1.252 rows=21.00 loops=1)
  Buffers: shared hit=99
  -> Sort (cost=331.38..331.63 rows=100 width=118) (actual time=1.245..1.247 rows=21.00 loops=1)
    Sort Key: created_at DESC, id DESC
    Sort Method: top-N heapsort Memory: 30kB
    Buffers: shared hit=99
    -> Bitmap Heap Scan on commandes (cost=5.07..328.68 rows=100 width=118) (actual
        time=0.280..1.153 rows=100.00 loops=1)
        Recheck Cond: (client_id = 42)
        Heap Blocks: exact=88
        Buffers: shared hit=93
        -> Bitmap Index Scan on a3_idx_simple (cost=0.00..5.04 rows=100 width=0) (actual
            time=0.129..0.130 rows=100.00 loops=1)
            Index Cond: (client_id = 42)
            Index Searches: 1
            Buffers: shared hit=5
Planning:
  Buffers: shared hit=150
Planning Time: 0.854 ms
Execution Time: 1.306 ms
```

### Composé

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z") AS
    created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit (cost=0.42..84.25 rows=21 width=118) (actual time=0.154..0.291 rows=21.00 loops=1)
  Buffers: shared hit=27
  -> Index Scan using a3_idx_compose on commandes (cost=0.42..399.64 rows=100 width=118) (actual
        time=0.152..0.286 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=160
Planning Time: 0.812 ms
Execution Time: 0.321 ms
```

## Couvrant (statut, total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit (cost=0.42..1.94 rows=21 width=118) (actual time=0.168..0.190 rows=21.00 loops=1)
  Buffers: shared hit=7
  -> Index Only Scan using a3_idx_couvrant on commandes (cost=0.42..7.67 rows=100 width=118) (actual
        time=0.166..0.187 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Heap Fetches: 0
        Index Searches: 1
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=167
Planning Time: 0.580 ms
Execution Time: 0.217 ms
```

## Couvrant (total)

```
-- SELECT id::text AS id, to_char(created_at AT TIME ZONE 'UTC','YYYY-MM-DD"T"HH24:MI:SS.US"Z"') AS
      created_at, statut, total::text AS total FROM shopflow.commandes AS commandes WHERE client_id = 42
      ORDER BY commandes.created_at DESC, commandes.id DESC LIMIT 21
Limit (cost=0.42..84.25 rows=21 width=118) (actual time=0.108..0.226 rows=21.00 loops=1)
  Buffers: shared hit=27
  -> Index Scan using a3_idx_couvrant_total on commandes (cost=0.42..399.64 rows=100 width=118)
        (actual time=0.107..0.223 rows=21.00 loops=1)
        Index Cond: (client_id = 42)
        Index Searches: 1
        Buffers: shared hit=27
Planning:
  Buffers: shared hit=163
Planning Time: 0.512 ms
Execution Time: 0.248 ms
```